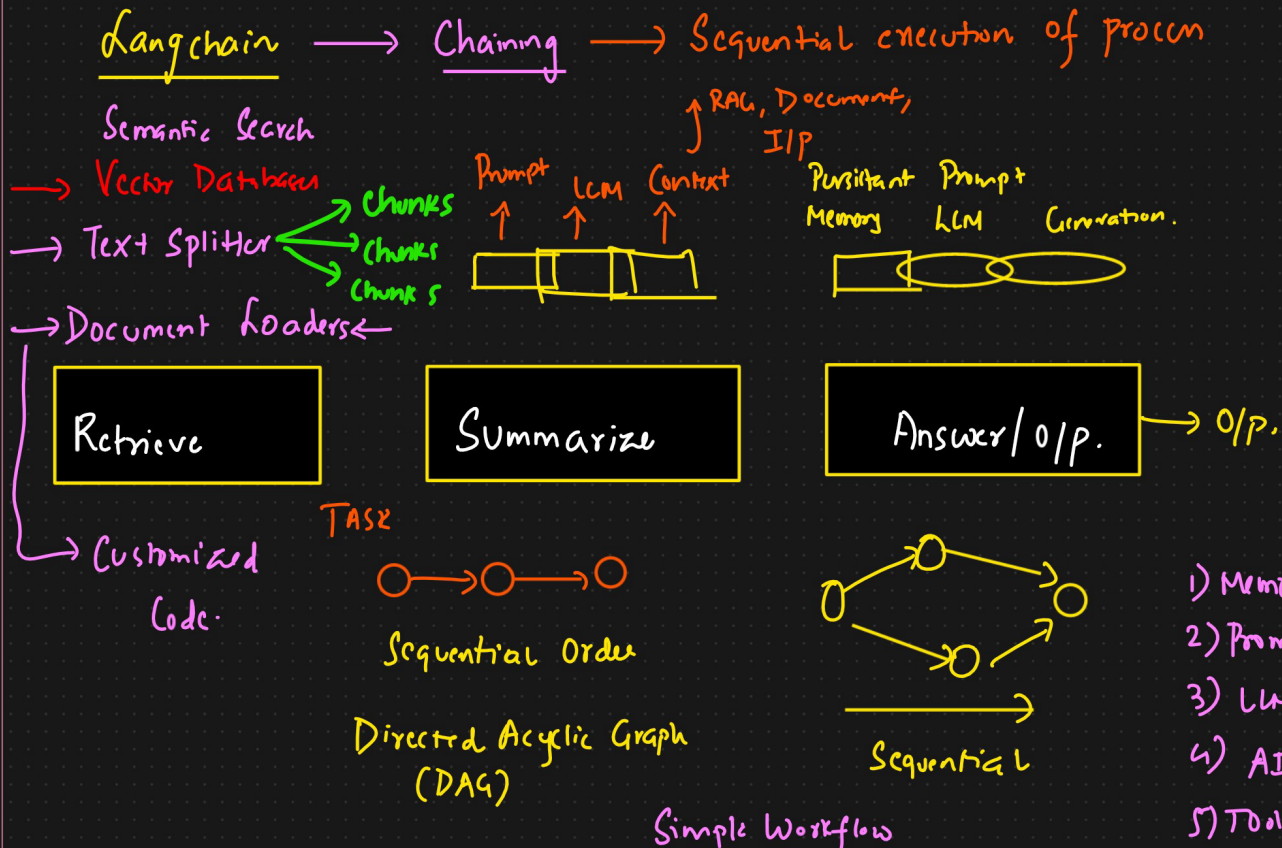


Agenda

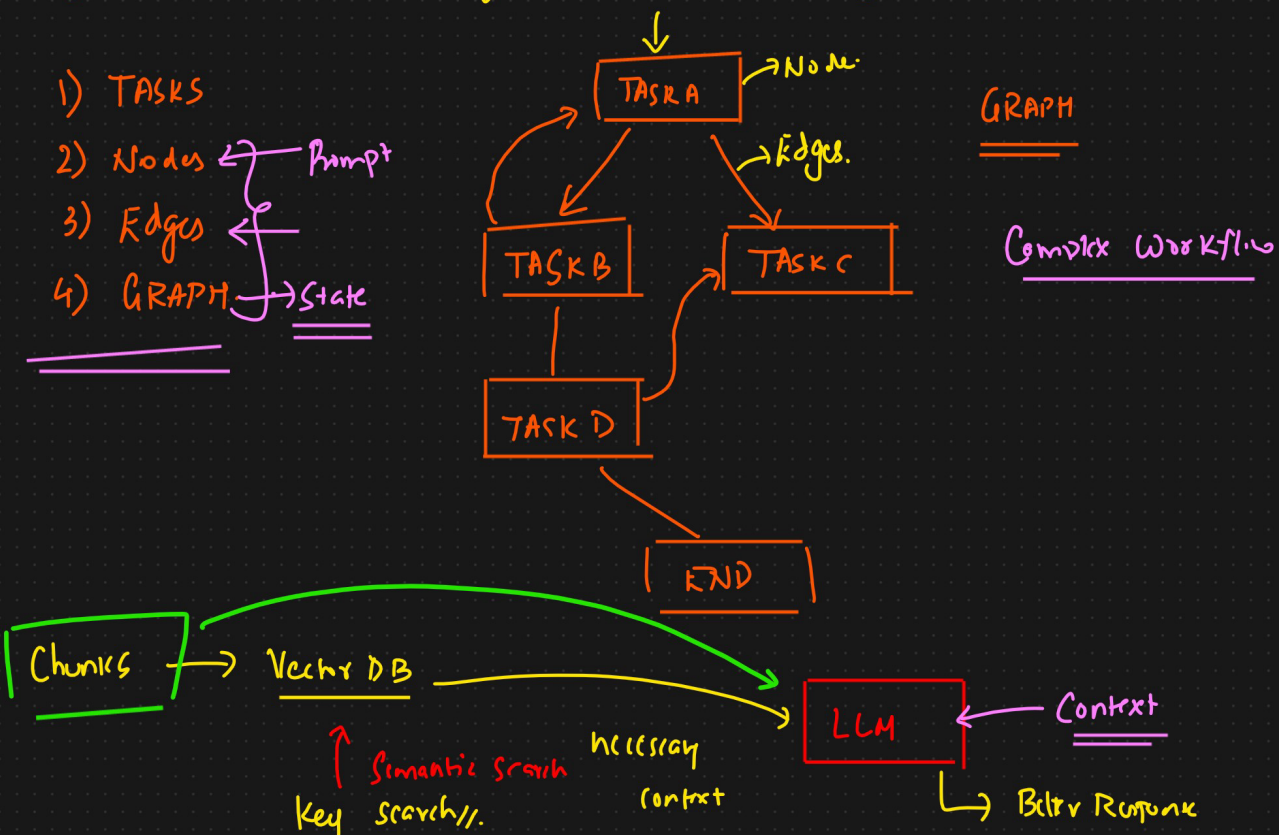
1) Langchain Vs LangGraph

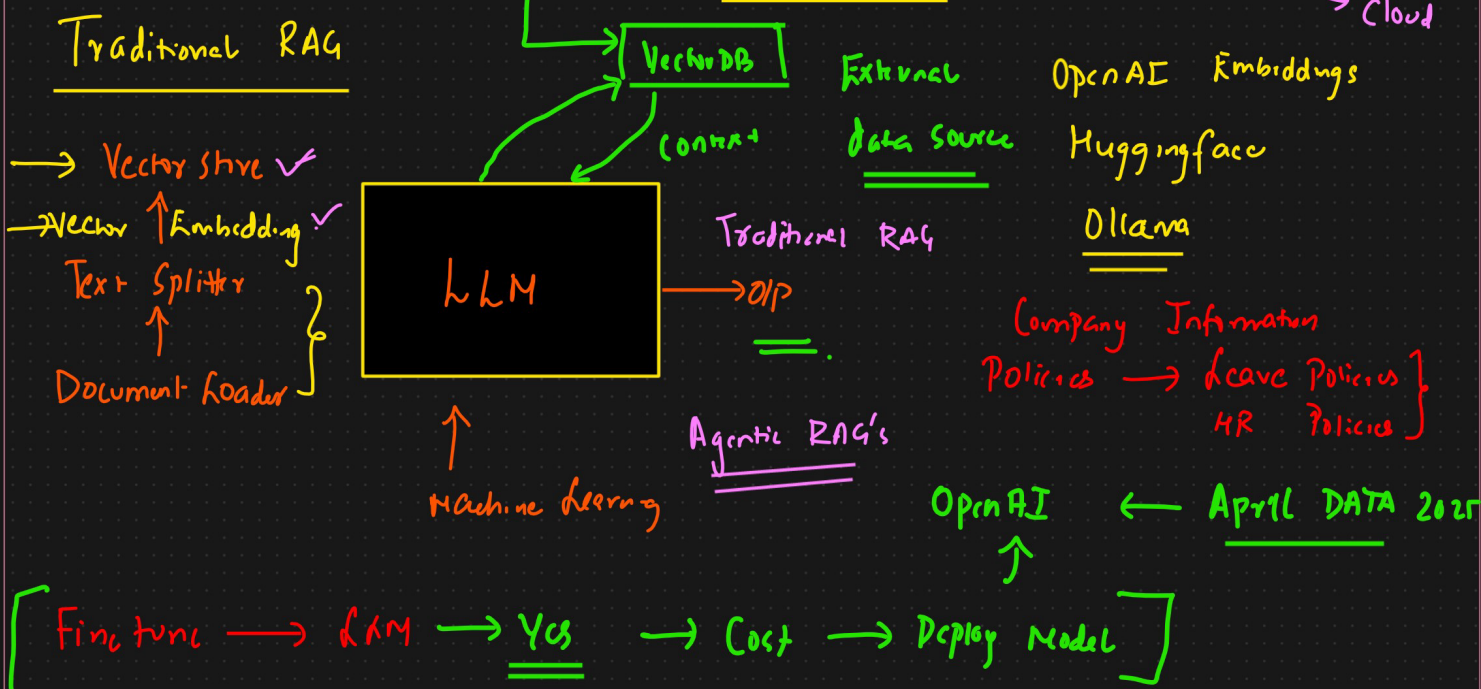
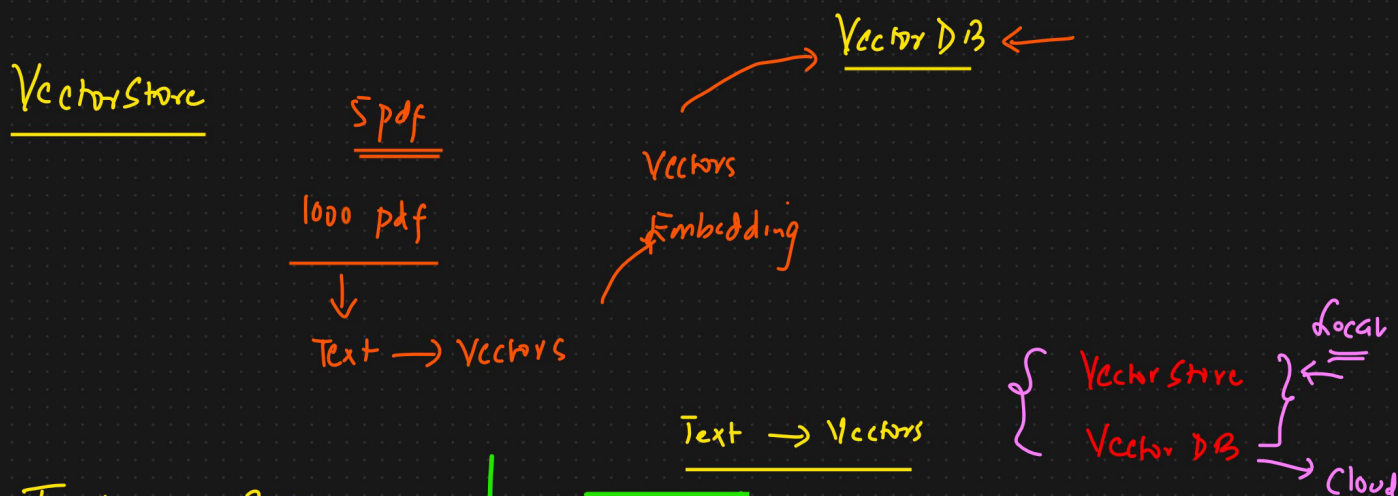
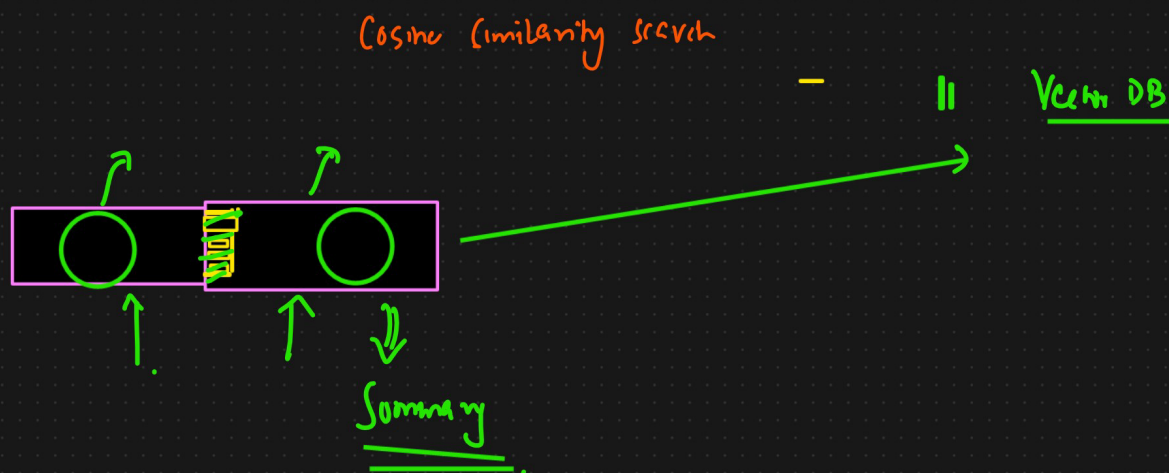
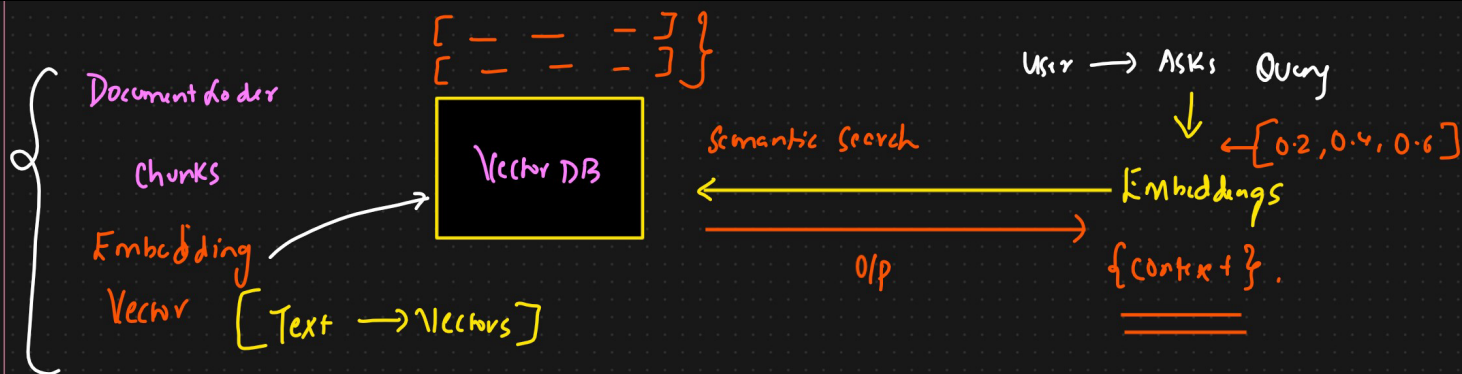
LLM => context

Website
pdfs, excel,
xml - - -



LangGraph : Stateful Multi AI Agents → Communicating → Complex Workflow.





RAG → Retrieval Augmented Generation.

Fine Tuning =
ChatBot ÷ Hi

Hi Welcome to KRISHNA ACADEMY

What would you like to learn?

GROK ←

Prompts → Prompt Template →

"Prompt" → System

"Fine Tuning" ⇒ Domain ← Company

"RAG" ⇒ External → keeps on changing

→ RAG
→ Tools
→ External API's

Imp.

★ Main Building Blocks in LangChain

Stage	Components / Purpose
Retrieve	- Document Loaders → bring data from PDFs, CSVs, web, DBs, etc. - Text Splitters → chunk text into smaller pieces (to fit context window). - Vector Stores → embed chunks + store them for semantic search.
Summarize	- Chains or agents that condense long documents into short, useful summaries. - Approaches: map-reduce, refine, stuff.
OP (Output Parsing)	- Output Parsers define the structure of LLM responses. - Examples: StrOutputParser, JsonOutputParser, PydanticOutputParser. - Ensures raw LLM text → structured data for your app.

why chunking

↳ Because there is a limit to Context Window

v.v. Imp.

◆ What LangChain gives you

LangChain is a **framework** — think of it as a *toolbox* to connect LLMs with data, prompts, workflows, and apps.

It provides building blocks like:

- Document loaders → load PDFs, CSVs, websites, DBs
- Text splitters → break big docs into chunks
- Embeddings + Vector Stores → semantic search
- Chains/Agents → orchestrate workflows
- Output Parsers → structure results

LangChain vs LangGraph

Aspect	LangChain (chain of Tasks)	LangGraph (agent workflows)
Focus	General LLM apps (RAG, chatbots, pipelines)	Agentic AI (stateful, multi-step agents)
Structure	Linear <u>chains</u> of prompts, tools, and models	<u>Graphs</u> of nodes & edges (state machines)
State	Stateless by default	Stateful by design
When to Use	- Quick RAG apps - Prompt chaining - Lightweight chatbots.	- Autonomous multi-step agents - Tool-using assistants - Applications needing reliability & state.

Q: what is difference b/w Stateless & Stateful?

Stateless

- The system does **not** remember past interactions.
- Each request is **independent**.
- If you ask the same question twice, it gives an answer without "knowing" what was asked before.

✓ Example:

A calculator app → every calculation is fresh:

```
2 + 2 = 4
5 + 5 = 10
```

It doesn't remember the previous calculation.

Stateful

- The system **remembers** past interactions (state) and uses them in the next step.
- Each new request depends on the **history / context**.

✓ Example:

A chat app with memory:

```
vbnet
Copy code

You: Who is Elon Musk?
AI: He is the CEO of Tesla and SpaceX.
You: Where was he born?
AI: Elon Musk was born in Pretoria, South Africa.
```

Here the AI remembers "he" refers to Elon Musk — that's being stateful.

Imp.

In your context (LangChain vs LangGraph):

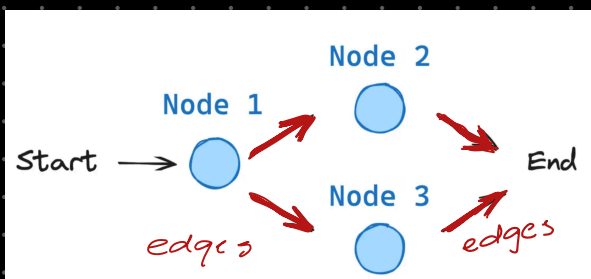
- LangChain → Stateless by default (you must add memory if you want the model to remember).
- LangGraph → Stateful by design (the graph naturally carries forward the agent's state across steps).

Lang Graph is based on the idea of DAG

DAG = Directed Acyclic Graph

↳ Graph of Nodes & edges

Blueprint
for
Lang Graph



Node = Each Task

Edge = Connects 2 Nodes

DAG = Entire process

Whole Agent = DAG

Components of Lang Graph

◆ 1. Each Node = a step → Each step (Task)

In LangGraph, a **node** is like a **block of work**.

Examples of nodes:

- **LLM Call Node** → calls GPT to generate text.
- **Tool Call Node** → fetches data from a database or API.
- **Decision Node** → chooses which path to follow (like "Do I need web search?").
- **Validation Node** → checks if output meets conditions (e.g., JSON format).

So every node is a **unit of logic**.

◆ 2. Each Edge = flow of control

→ Connects two Nodes Together

An **edge** connects one node to the next.

- If Node A finishes, an edge can point to Node B.
- Sometimes, **conditional edges** are used → if LLM says "Yes", go to Tool Node; if "No", go to Answer Node.

So edges decide **how information and execution moves**.

◆ 3. The whole Agent = a DAG

When you connect nodes with edges:

- The graph is **directed** (flows forward only).
- It's **acyclic** (no infinite loops).
- The **entire workflow/agent** is just this **DAG**.

Acyclic means no infinite loops.

◆ Example: Agent as DAG

less

```
[User Input] → [Decision Node: Need Tool?]  
  | Yes → [Tool Call Node] → [LLM Response Node] → [Output]  
  | No  → [LLM Response Node] → [Output]
```

- **Nodes:** Input, Decision, Tool, LLM, Output.
- **Edges:** connect them in sequence and conditionally.
- The **agent's logic** is nothing but this DAG.

COMPONENTS OF LAGCHAIN;

1. RETRIEVE

- Document loaders
- Text splitting
- Vector storage

2. SUMMARY

- chains
- context
- RAG
- memory

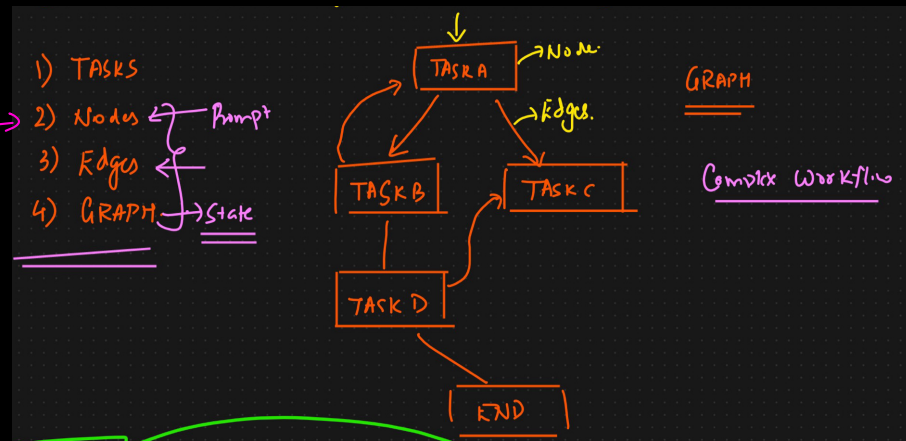
3. O/P

parsers

LANG GRAPH; Multi AI Agents / Workflows

Nodes $\longrightarrow$ Edges $\longrightarrow$ o/p { Acyclic

each node will have prompts.



Imp. Memory is efficient in Lang Graph.